Министерство образования Республики Беларусь

Учреждение образования

Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет

Кафедра экономической информатики

Отчет

по лабораторной работе № 3

на тему «Реализация базовой структуры системы и GraphQL API»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа продолжает архитектуру, спроектированную в ЛР1 и детализированную в ЛР2. На этом этапе архитектурная модель переводится в рабочую базовую реализацию. Сохраняются ранее определенные границы сервисов, отдельные базы данных и внешний GraphQL API. Реальные gRPC-вызовы и Kafka на данном этапе намеренно не используются. Межсервисные границы представлены интерфейсами и mock-клиентами, чтобы следующая лабораторная могла заменить заглушки на сетевое взаимодействие без переноса основной логики.

Цель работы - реализовать базовую структуру распределенной системы, подключить базы данных сервисов, реализовать модели и операции чтения, создать GraphQL Query на основе схемы ЛР2 и обеспечить mock-взаимодействие GraphQL Gateway с внутренними сервисами.

1. Выбранная реализация

Для реализации выбран Python. Внешний API построен на FastAPI и Strawberry GraphQL, работа с данными реализована через SQLAlchemy. В лабораторной версии для каждого сервиса используется отдельный файл SQLite. Это позволяет запустить проект без отдельного сервера СУБД и при этом сохранить принцип Database per Service.

UUID в SQLite хранятся как строки длиной 36 символов. Денежные значения хранятся через Numeric и в сервисном слое представлены Decimal. При переходе на PostgreSQL эти поля могут быть заменены на нативный UUID и NUMERIC без изменения внешнего GraphQL API.

Компонент	**Реализация**
GraphQL API	FastAPI + Strawberry GraphQL
Работа с БД	SQLAlchemy 2.x
Хранилища	7 отдельных SQLite БД
Межсервисное взаимодействие	Mock-клиенты через интерфейсы
Тестирование	Pytest

2. Структура проекта

GraphQL Gateway и внутренние сервисы физически разделены по каталогам. Внутри каждого сервиса отдельно находятся модели данных, репозиторий, подключение к собственной базе и сервисный слой. GraphQL resolver не получает доступ к SQLAlchemy Session и не импортирует таблицы внутренних сервисов.

Рисунок структуры каталогов проекта

```
lab3_profitability_system/
├── app/
│   ├── gateway/
│   │   ├── clients/
│   │   ├── facade.py
│   │   ├── main.py
│   │   └── schema.py
│   ├── services/
│   │   ├── identity/
│   │   ├── master_data/
│   │   ├── production/
│   │   ├── financial/
│   │   ├── costing/
│   │   ├── analytics/
│   │   └── reporting/
│   ├── shared/
│   ├── scripts/seed.py
│   └── bootstrap.py
├── docs/
├── tests/
├── requirements.txt
└── README.md
```

3. Архитектура базовой реализации

Запрос приходит от клиента в GraphQL Gateway. Resolver передает управление QueryFacade. Фасад работает только с интерфейсами клиентов. В ЛР3 этим интерфейсам соответствуют mock-реализации, которые вызывают сервисный слой в том же процессе. Внутренний сервис обращается к своему репозиторию, а репозиторий работает только с собственной базой данных.

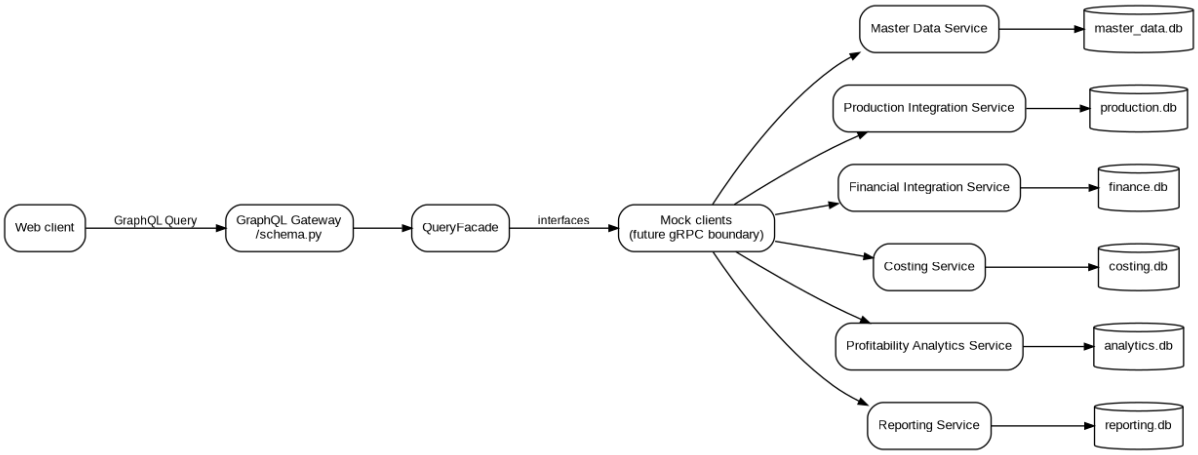


Рисунок 1. Архитектура ЛР3 с mock-взаимодействием

4. Внутренние сервисы и базы данных

Состав внутренних сервисов сохранен из ЛР1 и ЛР2. Каждый сервис отвечает только за свою область и подключается к собственной базе. Прямой доступ одного сервиса к таблицам другого отсутствует.

Сервис	**База**	**Ответственность**
Identity Service	identity.db	Пользователи и роли
Master Data Service	master_data.db	Продукция, группы, материалы, центры затрат
Production Integration Service	production.db	Производственные данные и журнал импорта
Financial Integration Service	finance.db	Реализация, финансовые записи и журнал импорта
Costing Service	costing.db	Расчеты себестоимости и распределения
Profitability Analytics Service	analytics.db	Рентабельность и статусы расчетов
Reporting Service	reporting.db	Read model, сохраненные отчеты и выгрузки

4.1 Слои внутреннего сервиса

Внутренний сервис разбит на несколько слоев. models.py содержит SQLAlchemy-модели. database.py создает подключение только к базе этого сервиса. repository.py реализует SQL-запросы. service.py преобразует строки БД в DTO и предоставляет бизнес-операции. Благодаря этому GraphQL слой не зависит от структуры таблиц.

Пример сервисного слоя Costing Service

```
class CostingService:
    def get_cost_breakdown(self, product_id, period, version=None):
        row = self.repository.find_cost_breakdown(
            product_id, period, version
        )
        if row is None:
            return None
        return CostBreakdownDTO(...)
```

5. Реализация GraphQL API

В ЛР3 реализуются только Query, то есть операции чтения. Mutation из ЛР2 в коде этой работы не реализуются, поскольку задание текущего этапа требует базовые сценарии получения данных. Схема построена от пользовательских сценариев, а не как прямой CRUD над таблицами.

Листинг 1. GraphQL Query, перенесенные из ЛР2

```
type Query {
  products(filter: ProductFilterInput): [Product!]!
  importRuns(source: ImportSource, period: Date): [ImportRun!]!
  costBreakdown(productId: ID!, period: Date!, version: Int): CostBreakdown
  profitability(filter: ProfitabilityFilterInput!): [ProfitabilityResult!]!
  calculationStatus(id: ID!): CalculationJob
  reportExport(id: ID!): ReportExport
}
```

5.1 Назначение Query

Query	**Назначение**
products(filter)	Получить список продукции с фильтром по группе, активности и строке поиска.
importRuns(source, period)	Получить историю загрузок производственного или финансового контура.
costBreakdown(productId, period, version)	Получить структуру себестоимости продукта.
profitability(filter)	Получить результаты рентабельности за период.
calculationStatus(id)	Получить состояние фонового расчета.
reportExport(id)	Получить состояние и ссылку на сформированную выгрузку.

5.2 Resolver не содержит бизнес-логику

Resolver принимает параметры GraphQL, передает их в QueryFacade и преобразует DTO в GraphQL-типы. Он не выполняет SQL и не рассчитывает показатели. Например, costBreakdown обращается к фасаду, фасад вызывает CostingClient, mock-клиент вызывает CostingService, а тот уже работает через репозиторий.

Листинг 2. Упрощенный resolver costBreakdown

```
@strawberry.field
def cost_breakdown(self, info, product_id, period, version=None):
    row = info.context["facade"].cost_breakdown(
        str(product_id), period, version
    )
    if row is None:
        return None
    return CostBreakdownType(...)
```

6. Mock-взаимодействие между компонентами

Вместо реального gRPC используются интерфейсы клиентов и временные mock-реализации. Это важная часть ЛР3: архитектура уже содержит будущую границу межсервисного вызова, но сетевой транспорт еще отсутствует. Mock-клиент не читает чужую базу, а вызывает публичный метод соответствующего внутреннего сервиса.

Листинг 3. Mock-клиент Costing Service

```
class MockCostingClient:
    def __init__(self, services):
        self.service = services.costing


    def get_cost_breakdown(self, product_id, period, version=None):
        return self.service.get_cost_breakdown(
            product_id, period, version
        )
```

В следующей лабораторной этот класс можно заменить, например, на GrpcCostingClient, который реализует тот же интерфейс и выполняет реальный сетевой вызов. QueryFacade и GraphQL resolver при этом остаются практически без изменений.

7. Базовая бизнес-логика

Для списков реализована фильтрация и возврат пустого списка, если подходящих данных нет. Для одиночных объектов costBreakdown, calculationStatus и reportExport при отсутствии записи возвращается null, что соответствует nullable-типам в GraphQL-схеме ЛР2. Репозитории не выбрасывают ошибки при обычном отсутствии данных.

Query importRuns маршрутизируется по источнику. ERP и MES относятся к Production Integration Service, FINANCE относится к Financial Integration Service. Если source не указан, фасад объединяет результаты обоих интеграционных контуров. Это координация сценария на уровне приложения, но не прямой доступ к двум базам из resolver.

8. Тестовые данные

Для проверки подготовлен скрипт app/scripts/seed.py. Он создает таблицы всех сервисов и записывает согласованный набор тестовых данных. В примере есть два продукта, завершенные производственный и финансовый импорты, версия себестоимости и рассчитанный результат рентабельности.

Показатель	**Тестовое значение**
Продукт	Пульт управления ПД-1
Период	01.09.2026
Материальные затраты	1850.00
Трудовые затраты	1350.00
Накладные расходы	800.00
Полная себестоимость	4000.00
Выпуск	100 шт.
Себестоимость единицы	40.0000
Выручка	6400.00
Прибыль	3200.00
Рентабельность	50.00 %

9. Примеры GraphQL-запросов

Листинг 4. Получение продукции

```
query Products {
  products(filter: { active: true }) {
    id
    code
    name
    unit
  }
}
```

Листинг 5. Получение рентабельности

```
query Profitability {
  profitability(filter: {
    productIds: ["22222222-2222-2222-2222-222222222222"]
    periodFrom: "2026-09-01"
    periodTo: "2026-09-30"
  }) {
    productId
```

```
    revenue
    costOfSales
    profit
    marginPercent
  }
}
```

Листинг 6. Получение структуры себестоимости

```
query CostBreakdown {
  costBreakdown(
    productId: "22222222-2222-2222-2222-222222222222"
    period: "2026-09-01"
  ) {
    materialCost
    laborCost
    overheadCost
    totalCost
    unitCost
  }
}
```

10. Запуск проекта

После распаковки проекта необходимо создать виртуальное окружение и установить зависимости из requirements.txt. Затем выполняется скрипт наполнения тестовыми данными и запускается FastAPI-приложение.

Команды запуска

```
python -m venv .venv
pip install -r requirements.txt
python -m app.scripts.seed
uvicorn app.gateway.main:app --reload
```

После запуска GraphQL UI доступен по пути /graphql, а проверка состояния приложения по /health. В каталоге docs находится отдельный файл graphql_examples.graphql с готовыми запросами для проверки.

11. Проверка реализации

Для сервисного слоя и QueryFacade подготовлены автоматические тесты. Проверяются чтение продуктов, получение себестоимости, расчетные данные рентабельности, корректная обработка отсутствующей записи, статусы расчета и выгрузки, а также маршрутизация importRuns через mock-клиенты.

Результат запуска pytest

```
........                                                                 [100%]
8 passed in 0.27s
```

GraphQL-слой синтаксически подготовлен на Strawberry. В архиве присутствует requirements.txt, поэтому после установки зависимостей API запускается стандартной командой uvicorn. В текущем окружении артефакт дополнительно проверен компиляцией Python-модулей, а сервисный слой и mock-взаимодействие проверены тестами.

Заключение

В лабораторной работе реализована базовая версия распределенной системы анализа рентабельности продукции. Архитектура из ЛР1 и ЛР2 перенесена в структуру кода: GraphQL API выделен отдельно, внутренние сервисы имеют собственные модели, репозитории, сервисный слой и отдельные базы данных. Реализованы основные Query для чтения продукции, истории импортов, себестоимости, рентабельности, статусов расчетов и выгрузок.

Межсервисное взаимодействие на данном этапе выполнено через mock-клиенты, что соответствует заданию ЛР3. Прямого доступа к чужим базам нет, а GraphQL resolver не содержит основную бизнес-логику. Полученная структура готова к следующему этапу, где mock-клиенты могут быть заменены реальными gRPC-вызовами, а система дополнена Kafka и операциями изменения данных.